Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 03

Prof. Claudio Benossi

1.
Unidade

Vetores, Matrizes, TAD e Alocação Dinâmica de Memória

Arranjos – Array (Vetores ou Matrizes)

Pessoal em nossos exemplos anteriores estamos trabalhando com um vetor do tipo **String**, porém podemos fazer uma simples modificação em nosso código que permite criar nosso vetor com diferentes tipos de dados, basta passar o parâmetro **String** para **Object**.

Arranjos – Array (Vetores ou Matrizes)

Nosso código conforme a aula anterior:

Vetor.java | Teste.java

Código-Fonte | Histórico

```java
1  package projeto_revisao_ed;
2
4  import java.util.Arrays;
4
5  public class Vetor {
6      private String[] elementos;
7      private int tamanho;
8
```

Modificar:

Nesse caso seria necessário alguns Ajustes ...

Vetor.java | Teste.java

Código-Fonte | Histórico

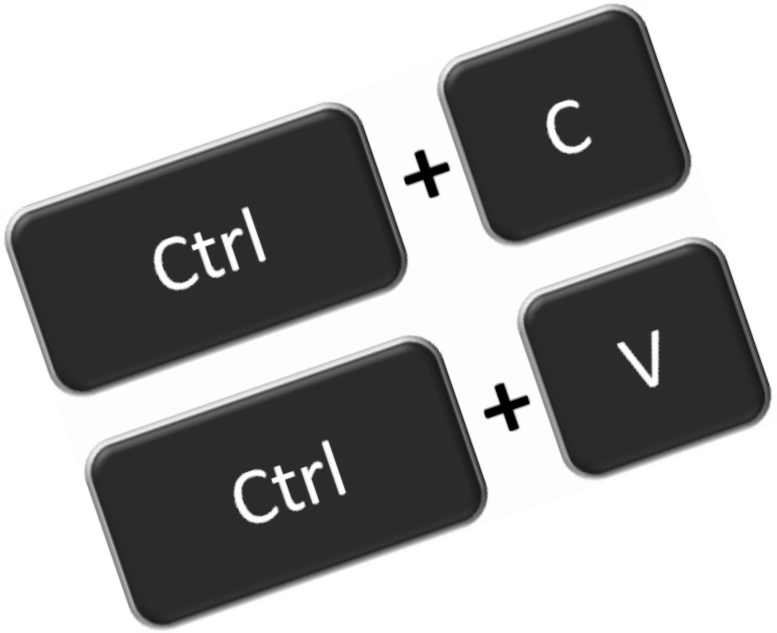
```java
package projeto_revisao_ed;

import java.util.Arrays;

public class Vetor {
    private Object[] elementos;
    private int tamanho;

    public Vetor(int capacidade){
        this.elementos = new Object[capacidade];
        this.tamanho = 0;
    }

    public void adiciona(Object elemento)throws Exception{
      this.aumentaCapacidade();
      if (this.tamanho < this.elementos.length){
          this.elementos[this.tamanho] = elemento;
          this.tamanho++;
      } else {
          throw new Exception("O Vetor já está cheio, "
                  + "não é possível adiconar novos elementos");
      }
    }

    public int tamanho(){
        return this.tamanho;
    }

```

Arranjos – Array (Vetores ou Matrizes)

Agora você pode testar passando como parâmetros diferentes tipos de dados...

Arranjos – Array (Vetores ou Matrizes)

OK, então vamos criar uma nova classe em nosso pacote com o nome de VetorObjeto que a princípio é uma cópia da nossa classe Vetor, porém alterando o parâmetro String para Object.



Projeto_Revisao_ED.java × Vetor.java × VetorObjeto.java ×

Source History

```java
package projeto_revisao_ed;

import java.util.Arrays;

public class VetorObjeto {
  // Lista de Atributos
    public Object[] elementos;
    public int tamanho;

    // Lista de Métodos
    public VetorObjeto(int capacidade) {
        this.elementos = new Object[capacidade];
        this.tamanho = 0;
    }

    public void adiciona(Object elemento) throws Exception {
        this.aumentaCapacidade();
        this.elementos[this.tamanho] = elemento;
        this.tamanho++;
    }

    public int tamanho() {
        return this.tamanho;
    }
```

Projeto_Revisao_ED.java × Vetor.java × VetorObjeto.java ×

Source History

```java
25
26      @Override
27      public String toString() {
28          StringBuilder s = new StringBuilder();
29          s.append("[");
30
31          for (int i = 0; i < this.tamanho - 1; i++) {
32              s.append(this.elementos[i]);
33              s.append(", ");
34          }
35
36          if (this.tamanho > 0) {
37              s.append(this.elementos[this.tamanho - 1]);
38          }
39
40          s.append("]");
41
42          return s.toString();
43      }
44
45      public Object busca(int posicao) throws Exception {
46          if (posicao >= 0 && posicao < tamanho) {
47              return this.elementos[posicao];
48          } else {
49              throw new Exception("Posicao Invalida");
50          }
51      }
```

Projeto_Revisao_ED.java × Vetor.java × VetorObjeto.java ×

Source History

```java
52
53      public int buscaI (Object elemento){
54          for (int i=0; i < tamanho; i++){
55              if(elementos[i].equals(elemento)){
56                  return i;
57              }
58          }
59          return -1;
60      }
61
62      public boolean adicionaInicio(int posicao, Object elemento) throws Exception {
63          this.aumentaCapacidade();
64          if (posicao >= 0 && posicao < tamanho) {
65              for (int i = this.tamanho - 1; i >= posicao; i--) {
66                  this.elementos[i + 1] = this.elementos[i];
67              }
68              this.elementos[posicao] = elemento;
69              this.tamanho++;
70          }
71          return true;
72      }
```

Projeto_Revisao_ED.java × Vetor.java × VetorObjeto.java ×

Source History

```java
    private void aumentaCapacidade(){
        if (this.tamanho == this.elementos.length){
            Object[] elementosNovos = new Object[this.elementos.length * 2];
            for (int i=0; i < this.elementos.length;i++){
                elementosNovos[i] = this.elementos[i];
            }
            this.elementos = elementosNovos;
        }
    }

    public void remove(int posicao) throws Exception{
        if(posicao >= 0 && posicao < tamanho){
            for (int i = posicao; i < this.tamanho-1; i++){
                this.elementos[i] = this.elementos[i+1];
            }
            this.elementos[tamanho] = null;
            this.tamanho--;
        } else {
            throw new Exception("Posição Inválida");
        }
    }

}
```

Arranjos – Array (Vetores ou Matrizes)

Agora para melhorar nosso exemplo, vamos criar uma classe **Contado**, contendo os atributos nome, telefone e email, todos do tipo String, será necessário criar também os métodos de acesso **Get** e **Set**, assim como o **construtor** e um método **toString** para organizar os dados na hora da exibição dos dados.

1º parte

Vetor.java | Teste.java | VetorObjeto.java | Contato.java

Código-Fonte | Histórico

```java
package projeto_revisao_ed;

public class Contato {
    private String nome;
    private String telefone;
    private String email;

    public Contato() {
    }

    public Contato(String nome, String telefone, String email) {
        this.nome = nome;
        this.telefone = telefone;
        this.email = email;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
```

2º parte

```
24
25      public String getTelefone() {
26          return telefone;
27      }
28
29      public void setTelefone(String telefone) {
30          this.telefone = telefone;
31      }
32
33      public String getEmail() {
34          return email;
35      }
36
37      public void setEmail(String email) {
38          this.email = email;
39      }
40
41      @Override
        public String toString() {
43          return "Contato{" + "nome=" + nome + ", telefone=" + telefone + ", email=" + email + '}';
44      }
45
46  }
```

Arranjos – Array (Vetores ou Matrizes)

Agora para testar, vamos criar alguns contatos, chamar os métodos para verificar o tamanho do vetor e para exibir os dados inseridos.

Vetor.java | Teste.java | VetorObjeto.java | Contato.java

Código-Fonte | Histórico

```java
package projeto_revisao_ed;

public class Teste {
    public static void main(String[] args) throws Exception{
        VetorObjeto vetor = new VetorObjeto(5);

        Contato c1 = new Contato("Claudio","9999-2525","claudio@gmail.com");
        Contato c2 = new Contato("Daniel","9999-7213","daniel@gmail.com");
        Contato c3 = new Contato("Miguel","9999-0101","miguel@gmail.com");

        try {
            vetor.adiciona(c1);
            vetor.adiciona(c2);
            vetor.adiciona(c3);
        } catch (Exception e) {
            e.printStackTrace();
        }

        System.out.println("Tamanho do vetor: " + vetor.tamanho());

        System.out.println(vetor);
    }
}
```

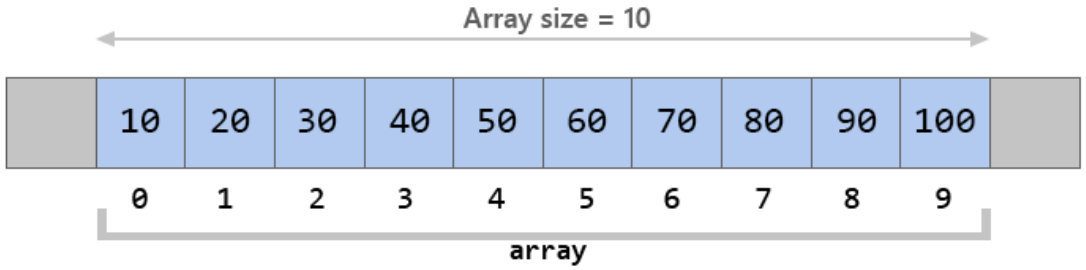
Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
Tamanho do vetor: 3
[Contato{nome=Claudio, telefone=9999-2525, email=claudio@gmail.com}, Contato{nome=Daniel, telefone=9999-7213, email=daniel@gmail.com}, Contato{nome=Miguel, telefone=9999-0101, email=migu
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)



Para encerrar o assunto Array, vamos falar sobre o **ArraList**, uma das funcionalidades mais utilizadas quando falamos em estrutura de dados com vetor.

Arranjos – Array (Vetores ou Matrizes)

Podemos dizer que **ArrayList** é uma classe para coleções.

Uma classe genérica (generic classes), para ser mais exato.

Coleções mesmo, de qualquer tipo de 'coisa' ou dado, não somente de tipos primitivos.

Arranjos – Array (Vetores ou Matrizes)

A classe **ArrayList** é uma implementação da interface List que utiliza um vetor para armazenar elementos.

O vetor interno da classe **ArrayList** é recriado quando há remoções de elemento, adições de elemento no fim da lista além da capacidade dimensionada e adições de elementos que não no final da lista.

Arranjos – Array (Vetores ou Matrizes)

A classe **ArrayList** já vem com vários métodos prontos na linguagem Java, vamos conferir alguns ...

Arranjos – Array (Vetores ou Matrizes)

Teste.java

Código-Fonte Histórico

```
1   package projeto_revisao_ed;
2
3   import java.util.ArrayList;
4
5   public class Teste {
6       public static void main(String[] args) throws Exception{
        ArrayList<String> arrayList = new ArrayList<String>();
8
9           arrayList.add("Elemento A");
10          arrayList.add("Elemento C");
11
12          System.out.println(arrayList);
13
14      }
15  }
16
```

Arranjos – Array (Vetores ou Matrizes)

O método **add** insere o elemento dentro do vetor, semelhante ao método **adicionar** que criamos nos exemplos anteriores.

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Também posso solicitar a adição de um novo elemento numa posição especifica e ele ajusta automaticamente a estrutura do nosso vetor.

Teste.java

Código-Fonte | Histórico

```java
1   package projeto_revisao_ed;
2
3   import java.util.ArrayList;
4
5   public class Teste {
6       public static void main(String[] args) throws Exception{
        ArrayList<String> arrayList = new ArrayList<String>();
8
9           arrayList.add("Elemento A");
10          arrayList.add("Elemento C");
11
12          System.out.println(arrayList);
13
14          arrayList.add(1, "Elemento B");
15
16          System.out.println(arrayList);
17
18      }
19  }
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Também podemos fazer buscas em nosso vetor com o método **contains**.

Teste.java

Código-Fonte | Histórico

```java
package projeto_revisao_ed;

import java.util.ArrayList;

public class Teste {
    public static void main(String[] args) throws Exception{
        ArrayList<String> arrayList = new ArrayList<String>();

        arrayList.add("Elemento A");
        arrayList.add("Elemento C");

        System.out.println(arrayList);

        arrayList.add(1, "Elemento B");

        System.out.println(arrayList);

        boolean existe = arrayList.contains("Elemento C");
        if (existe){
            System.out.println("Elementos encontrado no vetor");
        } else {
            System.out.println("Elementos não existe no vetor");
        }
    }
}
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

```
Saída - projeto_revisao_ed (run)
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
Elementos encontrado no vetor
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Caso eu queira encontrar a posição no elemento no vetor também é possível.

Teste.java

Código-Fonte | Histórico

```java
1  package projeto_revisao_ed;
2  import java.util.ArrayList;
3
4  public class Teste {
5      public static void main(String[] args) throws Exception{
6          ArrayList<String> arrayList = new ArrayList<String>();
7
8          arrayList.add("Elemento A");
9          arrayList.add("Elemento C");
10         System.out.println(arrayList);
11         arrayList.add(1, "Elemento B");
12
13         System.out.println(arrayList);
14
15         boolean existe = arrayList.contains("Elemento C");
16         if (existe){
17             System.out.println("Elementos encontrado no vetor");
18         } else {
19             System.out.println("Elementos não existe no vetor");
20         }
21
22         int posicao = arrayList.indexOf("Elemento C");
23         if (posicao > -1){
24             System.out.println("Elementos encontrado na posição nº " + posicao);
25         } else {
26             System.out.println("Elementos não existe no vetor");
27         }
28     }
29 }
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
Elementos encontrado no vetor
Elementos encontrado na posição nº 2
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Método de busca por posição arraList.get():

Teste.java

Código-Fonte | Histórico

```
1   package projeto_revisao_ed;
2   import java.util.ArrayList;
3
4   public class Teste {
5       public static void main(String[] args) throws Exception{
            ArrayList<String> arrayList = new ArrayList<String>();
7
8           arrayList.add("Elemento A");
9           arrayList.add("Elemento C");
10          System.out.println(arrayList);
11          arrayList.add(1, "Elemento B");
12
13          System.out.println(arrayList);
14
15          System.out.println(arrayList.get(1));
16
17      }
18  }
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
Elemento B
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Agora vamos **remover** um item do nosso **ArrayList** com o **método remove**, que pode ser usado por índice ou por objeto:

Teste.java

Código-Fonte | Histórico

```java
1  package projeto_revisao_ed;
2  import java.util.ArrayList;
3
4  public class Teste {
5      public static void main(String[] args) throws Exception{
6          ArrayList<String> arrayList = new ArrayList<String>();
7
8          arrayList.add("Elemento A");
9          arrayList.add("Elemento C");
10         System.out.println(arrayList);
11         arrayList.add(1, "Elemento B");
12
13         System.out.println(arrayList);
14
15         arrayList.remove(0);
16         arrayList.remove("Elemento C");
17
18         System.out.println(arrayList);
19     }
20 }
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
[Elemento B]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Para saber o tamanho do ArrayList podemos utilizar o método **size**:

Teste.java

Código-Fonte | Histórico

```java
1   package projeto_revisao_ed;
2   import java.util.ArrayList;
3
4   public class Teste {
5       public static void main(String[] args) throws Exception{
            ArrayList<String> arrayList = new ArrayList<String>();
7
8           arrayList.add("Elemento A");
9           arrayList.add("Elemento C");
10          System.out.println(arrayList);
11          arrayList.add(1, "Elemento B");
12
13          System.out.println(arrayList);
14
15          System.out.println("Tamanto do vetor: " + arrayList.size());
16      }
17  }
18
```

Arranjos – Array (Vetores ou Matrizes)

Resultado:

Saída - projeto_revisao_ed (run)

```
run:
[Elemento A, Elemento C]
[Elemento A, Elemento B, Elemento C]
Tamanto do vetor: 3
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Arranjos – Array (Vetores ou Matrizes)

Lembrando que ainda existem outros métodos que podemos utilizar no nosso dia-a-dia:

add(String e)	boolean
add(int index, String element)	void
addAll(Collection<? extends String> c)	boolean
addAll(int index, Collection<? extends String> c)	boolean
clear()	void
clone()	Object
contains(Object o)	boolean
containsAll(Collection<?> c)	boolean
ensureCapacity(int minCapacity)	void
equals(Object o)	boolean
forEach(Consumer<? super String> action)	void
get(int index)	String

getClass()	Class<?>
hashCode()	int
indexOf(Object o)	int
isEmpty()	boolean
iterator()	Iterator<String>
lastIndexOf(Object o)	int
listIterator()	ListIterator<String>
listIterator(int index)	ListIterator<String>
notify()	void
notifyAll()	void
parallelStream()	Stream<String>
remove(Object o)	boolean

.... Tem mais ....

Vamos pesquisar e descobrir para que serve cada um deles.

ADO – Atividade Discente Orientada

- Desenvolver uma aplicação utilizando a linguagem JAVA e os conceitos da Aula 02 e Aula 03, sobre Array e manipulação de dados.
- Descrever a situação e/ou problema (tema livre);
- Criar uma Classe Vetor com todos os métodos para manipular os dados;
- Criar uma Classe para testar a aplicação, possibilitando a interação com o usuário, para manipular os dados;

"A meta é se melhor que ontem,
NÃO melhor que ninguém!"

Autor desconhecido

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br